raylib

A simple and easy-to-use library to enjoy videogames programming

[raylib Discord server][github.com/raysan5/raylib][raylib.h]

v6.0 quick reference card [download as PDF]

Chinese Translation: 以下为raylib所有API接口中文释义

module: rcore

```
// Window-related functions
void InitWindow(int width, int height, const char *title);  // Initialize window and OpenGL context
void CloseWindow(void);                                     // Close window and unload OpenGL context
bool WindowShouldClose(void);                               // Check if application should close (KEY_ESCAPE pressed or windows close icon clicked)
bool IsWindowReady(void);                                   // Check if window has been initialized successfully
bool IsWindowFullscreen(void);                              // Check if window is currently fullscreen
bool IsWindowHidden(void);                                  // Check if window is currently hidden
bool IsWindowMinimized(void);                               // Check if window is currently minimized
bool IsWindowMaximized(void);                               // Check if window is currently maximized
bool IsWindowFocused(void);                                 // Check if window is currently focused
bool IsWindowResized(void);                                 // Check if window has been resized last frame
bool IsWindowState(unsigned int flag);                      // Check if one specific window flag is enabled
void SetWindowState(unsigned int flags);                    // Set window configuration state using flags
void ClearWindowState(unsigned int flags);                  // Clear window configuration state flags
void ToggleFullscreen(void);                                // Toggle window state: fullscreen/windowed, resizes monitor to match window resolution
void ToggleBorderlessWindowed(void);                        // Toggle window state: borderless windowed, resizes window to match monitor resolution
void MaximizeWindow(void);                                  // Set window state: maximized, if resizable
void MinimizeWindow(void);                                  // Set window state: minimized, if resizable
void RestoreWindow(void);                                   // Restore window from being minimized/maximized
void SetWindowIcon(Image image);                            // Set icon for window (single image, RGBA 32bit)
void SetWindowIcons(Image *images, int count);              // Set icon for window (multiple images, RGBA 32bit)
void SetWindowTitle(const char *title);                     // Set title for window
void SetWindowPosition(int x, int y);                       // Set window position on screen
void SetWindowMonitor(int monitor);                         // Set monitor for the current window
void SetWindowMinSize(int width, int height);               // Set window minimum dimensions (for FLAG_WINDOW_RESIZABLE)
void SetWindowMaxSize(int width, int height);               // Set window maximum dimensions (for FLAG_WINDOW_RESIZABLE)
void SetWindowSize(int width, int height);                  // Set window dimensions
void SetWindowOpacity(float opacity);                       // Set window opacity [0.0f..1.0f]
void SetWindowFocused(void);                                // Set window focused
void *GetWindowHandle(void);                                // Get native window handle
int GetScreenWidth(void);                                   // Get current screen width
int GetScreenHeight(void);                                  // Get current screen height
int GetRenderWidth(void);                                   // Get current render width (it considers HiDPI)
int GetRenderHeight(void);                                  // Get current render height (it considers HiDPI)
int GetMonitorCount(void);                                  // Get number of connected monitors
int GetCurrentMonitor(void);                                // Get current monitor where window is placed
Vector2 GetMonitorPosition(int monitor);                    // Get specified monitor position
int GetMonitorWidth(int monitor);                           // Get specified monitor width (current video mode used by monitor)
int GetMonitorHeight(int monitor);                          // Get specified monitor height (current video mode used by monitor)
int GetMonitorPhysicalWidth(int monitor);                   // Get specified monitor physical width in millimetres
int GetMonitorPhysicalHeight(int monitor);                  // Get specified monitor physical height in millimetres
int GetMonitorRefreshRate(int monitor);                     // Get specified monitor refresh rate
Vector2 GetWindowPosition(void);                            // Get window position XY on monitor
Vector2 GetWindowScaleDPI(void);                            // Get window scale DPI factor
const char *GetMonitorName(int monitor);                    // Get the human-readable, UTF-8 encoded name of the specified monitor
void SetClipboardText(const char *text);                    // Set clipboard text content
const char *GetClipboardText(void);                         // Get clipboard text content
```

```
Image GetClipboardImage(void);                          // Get clipboard image content
void EnableEventWaiting(void);                          // Enable waiting for events on EndDrawing(), no automatic event polling
void DisableEventWaiting(void);                         // Disable waiting for events on EndDrawing(), automatic events polling

// Cursor-related functions
void ShowCursor(void);                                  // Shows cursor
void HideCursor(void);                                  // Hides cursor
bool IsCursorHidden(void);                              // Check if cursor is not visible
void EnableCursor(void);                                // Enables cursor (unlock cursor)
void DisableCursor(void);                               // Disables cursor (lock cursor)
bool IsCursorOnScreen(void);                            // Check if cursor is on the screen

// Drawing-related functions
void ClearBackground(Color color);                      // Set background color (framebuffer clear color)
void BeginDrawing(void);                                // Setup canvas (framebuffer) to start drawing
void EndDrawing(void);                                  // End canvas drawing and swap buffers (double buffering)
void BeginMode2D(Camera2D camera);                      // Begin 2D mode with custom camera (2D)
void EndMode2D(void);                                   // Ends 2D mode with custom camera
void BeginMode3D(Camera3D camera);                      // Begin 3D mode with custom camera (3D)
void EndMode3D(void);                                   // Ends 3D mode and returns to default 2D orthographic mode
void BeginTextureMode(RenderTexture2D target);          // Begin drawing to render texture
void EndTextureMode(void);                              // Ends drawing to render texture
void BeginShaderMode(Shader shader);                    // Begin custom shader drawing
void EndShaderMode(void);                               // End custom shader drawing (use default shader)
void BeginBlendMode(int mode);                          // Begin blending mode (alpha, additive, multiplied, subtract, custom)
void EndBlendMode(void);                                // End blending mode (reset to default: alpha blending)
void BeginScissorMode(int x, int y, int width, int height); // Begin scissor mode (define screen area for following drawing)
void EndScissorMode(void);                              // End scissor mode
void BeginVrStereoMode(VrStereoConfig config);          // Begin stereo rendering (requires VR simulator)
void EndVrStereoMode(void);                             // End stereo rendering (requires VR simulator)

// VR stereo config functions for VR simulator
VrStereoConfig LoadVrStereoConfig(VrDeviceInfo device);     // Load VR stereo config for VR simulator device parameters
void UnloadVrStereoConfig(VrStereoConfig config);           // Unload VR stereo config

// Shader management functions
// NOTE: Shader functionality is not available on OpenGL 1.1
Shader LoadShader(const char *vsFileName, const char *fsFileName);   // Load shader from files and bind default locations
Shader LoadShaderFromMemory(const char *vsCode, const char *fsCode); // Load shader from code strings and bind default locations
bool IsShaderValid(Shader shader);                                   // Check if a shader is valid (loaded on GPU)
int GetShaderLocation(Shader shader, const char *uniformName);       // Get shader uniform location
int GetShaderLocationAttrib(Shader shader, const char *attribName);  // Get shader attribute location
void SetShaderValue(Shader shader, int locIndex, const void *value, int uniformType); // Set shader uniform value
void SetShaderValueV(Shader shader, int locIndex, const void *value, int uniformType, int count); // Set shader uniform value vector
void SetShaderValueMatrix(Shader shader, int locIndex, Matrix mat);  // Set shader uniform value (matrix 4x4)
void SetShaderValueTexture(Shader shader, int locIndex, Texture2D texture); // Set shader uniform value and bind the texture (sampler2d)
void UnloadShader(Shader shader);                                    // Unload shader from GPU memory (VRAM)

// Screen-space-related functions
Ray GetScreenToWorldRay(Vector2 position, Camera camera);         // Get a ray trace from screen position (i.e mouse)
Ray GetScreenToWorldRayEx(Vector2 position, Camera camera, int width, int height); // Get a ray trace from screen position (i.e mouse) in a viewport
Vector2 GetWorldToScreen(Vector3 position, Camera camera);        // Get the screen space position for a 3d world space position
Vector2 GetWorldToScreenEx(Vector3 position, Camera camera, int width, int height); // Get size position for a 3d world space position
Vector2 GetWorldToScreen2D(Vector2 position, Camera2D camera);    // Get the screen space position for a 2d camera world space position
Vector2 GetScreenToWorld2D(Vector2 position, Camera2D camera);    // Get the world space position for a 2d camera screen space position
Matrix GetCameraMatrix(Camera camera);                            // Get camera transform matrix (view matrix)
Matrix GetCameraMatrix2D(Camera2D camera);                        // Get camera 2d transform matrix

// Timing-related functions
void SetTargetFPS(int fps);                     // Set target FPS (maximum)
float GetFrameTime(void);                       // Get time in seconds for last frame drawn (delta time)
double GetTime(void);                           // Get elapsed time in seconds since InitWindow()
int GetFPS(void);                               // Get current FPS
```

```
// Custom frame control functions
// NOTE: Those functions are intended for advanced users that want full control over the frame processing
// By default EndDrawing() does this job: draws everything + SwapScreenBuffer() + manage frame timing + PollInputEvents()
// To avoid that behaviour and control frame processes manually, enable in config.h: SUPPORT_CUSTOM_FRAME_CONTROL
void SwapScreenBuffer(void);                    // Swap back buffer with front buffer (screen drawing)
void PollInputEvents(void);                     // Register all input events
void WaitTime(double seconds);                  // Wait for some time (halt program execution)

// Random values generation functions
void SetRandomSeed(unsigned int seed);          // Set the seed for the random number generator
int GetRandomValue(int min, int max);           // Get a random value between min and max (both included)
int *LoadRandomSequence(unsigned int count, int min, int max); // Load random values sequence, no values repeated
void UnloadRandomSequence(int *sequence);       // Unload random values sequence

// Misc. functions
void TakeScreenshot(const char *fileName);      // Takes a screenshot of current screen (filename extension defines format)
void SetConfigFlags(unsigned int flags);        // Setup init configuration flags (view FLAGS)
void OpenURL(const char *url);                  // Open URL with default system browser (if available)

// Logging system
void SetTraceLogLevel(int logLevel);            // Set the current threshold (minimum) log level
void TraceLog(int logLevel, const char *text, ...); // Show trace log messages (LOG_DEBUG, LOG_INFO, LOG_WARNING, LOG_ERROR...)
void SetTraceLogCallback(TraceLogCallback callback); // Set custom trace log

// Memory management, using internal allocators
void *MemAlloc(unsigned int size);              // Internal memory allocator
void *MemRealloc(void *ptr, unsigned int size); // Internal memory reallocator
void MemFree(void *ptr);                        // Internal memory free

// File system management functions
unsigned char *LoadFileData(const char *fileName, int *dataSize); // Load file data as byte array (read)
void UnloadFileData(unsigned char *data);           // Unload file data allocated by LoadFileData()
bool SaveFileData(const char *fileName, void *data, int dataSize); // Save data to file from byte array (write), returns true on success
bool ExportDataAsCode(const unsigned char *data, int dataSize, const char *fileName); // Export data to code (.h), returns true on success
char *LoadFileText(const char *fileName);           // Load text data from file (read), returns a '\0' terminated string
void UnloadFileText(char *text);                    // Unload file text data allocated by LoadFileText()
bool SaveFileText(const char *fileName, const char *text);    // Save text data to file (write), string must be '\0' terminated, returns true on success

// File access custom callbacks
// WARNING: Callbacks setup is intended for advanced users
void SetLoadFileDataCallback(LoadFileDataCallback callback);  // Set custom file binary data loader
void SetSaveFileDataCallback(SaveFileDataCallback callback);  // Set custom file binary data saver
void SetLoadFileTextCallback(LoadFileTextCallback callback);  // Set custom file text data loader
void SetSaveFileTextCallback(SaveFileTextCallback callback);  // Set custom file text data saver

int FileRename(const char *fileName, const char *fileRename); // Rename file (if exists)
int FileRemove(const char *fileName);               // Remove file (if exists)
int FileCopy(const char *srcPath, const char *dstPath);       // Copy file from one path to another, dstPath created if it doesn't exist
int FileMove(const char *srcPath, const char *dstPath);       // Move file from one directory to another, dstPath created if it doesn't exist
int FileTextReplace(const char *fileName, const char *search, const char *replacement); // Replace text in an existing file
int FileTextFindIndex(const char *fileName, const char *search); // Find text in existing file
bool FileExists(const char *fileName);              // Check if file exists
bool DirectoryExists(const char *dirPath);          // Check if a directory path exists
bool IsFileExtension(const char *fileName, const char *ext);  // Check file extension (recommended include point: .png, .wav)
int GetFileLength(const char *fileName);            // Get file length in bytes (NOTE: GetFileSize() conflicts with windows.h)
long GetFileModTime(const char *fileName);          // Get file modification time (last write time)
const char *GetFileExtension(const char *fileName);           // Get pointer to extension for a filename string (includes dot: '.png')
const char *GetFileName(const char *filePath);                // Get pointer to filename for a path string
const char *GetFileNameWithoutExt(const char *filePath);      // Get filename string without extension (uses static string)
const char *GetDirectoryPath(const char *filePath);           // Get full path for a given fileName with path (uses static string)
const char *GetPrevDirectoryPath(const char *dirPath);        // Get previous directory path for a given path (uses static string)
const char *GetWorkingDirectory(void);                        // Get current working directory (uses static string)
const char *GetApplicationDirectory(void);                    // Get the directory of the running application (uses static string)
int MakeDirectory(const char *dirPath);                       // Create directories (including full path requested), returns 0 on success
```

```
bool ChangeDirectory(const char *dirPath);                      // Change working directory, return true on success
bool IsPathFile(const char *path);                              // Check if a given path is a file or a directory
bool IsFileNameValid(const char *fileName);                     // Check if fileName is valid for the platform/OS
FilePathList LoadDirectoryFiles(const char *dirPath);           // Load directory filepaths, files and directories, no subdirs scan
FilePathList LoadDirectoryFilesEx(const char *basePath, const char *filter, bool scanSubdirs); // Load directory filepaths with extension filtering and subdir scan; some filters available: "*.*", "
void UnloadDirectoryFiles(FilePathList files);                  // Unload filepaths
bool IsFileDropped(void);                                       // Check if a file has been dropped into window
FilePathList LoadDroppedFiles(void);                            // Load dropped filepaths
void UnloadDroppedFiles(FilePathList files);                    // Unload dropped filepaths
unsigned int GetDirectoryFileCount(const char *dirPath);        // Get the file count in a directory
unsigned int GetDirectoryFileCountEx(const char *basePath, const char *filter, bool scanSubdirs); // Get the file count in a directory with extension filtering and recursive directory scan. Use 'DI

// Compression/Encoding functionality
unsigned char *CompressData(const unsigned char *data, int dataSize, int *compDataSize);        // Compress data (DEFLATE algorithm), memory must be MemFree()
unsigned char *DecompressData(const unsigned char *compData, int compDataSize, int *dataSize);  // Decompress data (DEFLATE algorithm), memory must be MemFree()
char *EncodeDataBase64(const unsigned char *data, int dataSize, int *outputSize);               // Encode data to Base64 string (includes NULL terminator), memory must be MemFree()
unsigned char *DecodeDataBase64(const char *text, int *outputSize);                             // Decode Base64 string (expected NULL terminated), memory must be MemFree()
unsigned int ComputeCRC32(unsigned char *data, int dataSize); // Compute CRC32 hash code
unsigned int *ComputeMD5(unsigned char *data, int dataSize);  // Compute MD5 hash code, returns static int[4] (16 bytes)
unsigned int *ComputeSHA1(unsigned char *data, int dataSize); // Compute SHA1 hash code, returns static int[5] (20 bytes)
unsigned int *ComputeSHA256(unsigned char *data, int dataSize); // Compute SHA256 hash code, returns static int[8] (32 bytes)

// Automation events functionality
AutomationEventList LoadAutomationEventList(const char *fileName); // Load automation events list from file, NULL for empty list, capacity = MAX_AUTOMATION_EVENTS
void UnloadAutomationEventList(AutomationEventList list);   // Unload automation events list from file
bool ExportAutomationEventList(AutomationEventList list, const char *fileName); // Export automation events list as text file
void SetAutomationEventList(AutomationEventList *list);     // Set automation event list to record to
void SetAutomationEventBaseFrame(int frame);                // Set automation event internal base frame to start recording
void StartAutomationEventRecording(void);                   // Start recording automation events (AutomationEventList must be set)
void StopAutomationEventRecording(void);                    // Stop recording automation events
void PlayAutomationEvent(AutomationEvent event);            // Play a recorded automation event

//------------------------------------------------------------------------------------
// Input Handling Functions (Module: core)
//------------------------------------------------------------------------------------

// Input-related functions: keyboard
bool IsKeyPressed(int key);                             // Check if a key has been pressed once
bool IsKeyPressedRepeat(int key);                       // Check if a key has been pressed again
bool IsKeyDown(int key);                                // Check if a key is being pressed
bool IsKeyReleased(int key);                            // Check if a key has been released once
bool IsKeyUp(int key);                                  // Check if a key is NOT being pressed
int GetKeyPressed(void);                                // Get key pressed (keycode), call it multiple times for keys queued, returns 0 when the queue is empty
int GetCharPressed(void);                               // Get char pressed (unicode), call it multiple times for chars queued, returns 0 when the queue is empty
const char *GetKeyName(int key);                        // Get name of a QWERTY key on the current keyboard layout (eg returns string 'q' for KEY_A on an AZERTY keyboard)
void SetExitKey(int key);                               // Set a custom key to exit program (default is ESC)

// Input-related functions: gamepads
bool IsGamepadAvailable(int gamepad);                   // Check if a gamepad is available
const char *GetGamepadName(int gamepad);                // Get gamepad internal name id
bool IsGamepadButtonPressed(int gamepad, int button);   // Check if a gamepad button has been pressed once
bool IsGamepadButtonDown(int gamepad, int button);      // Check if a gamepad button is being pressed
bool IsGamepadButtonReleased(int gamepad, int button);  // Check if a gamepad button has been released once
bool IsGamepadButtonUp(int gamepad, int button);        // Check if a gamepad button is NOT being pressed
int GetGamepadButtonPressed(void);                      // Get the last gamepad button pressed
int GetGamepadAxisCount(int gamepad);                   // Get axis count for a gamepad
float GetGamepadAxisMovement(int gamepad, int axis);    // Get movement value for a gamepad axis
int SetGamepadMappings(const char *mappings);           // Set internal gamepad mappings (SDL_GameControllerDB)
void SetGamepadVibration(int gamepad, float leftMotor, float rightMotor, float duration); // Set gamepad vibration for both motors (duration in seconds)

// Input-related functions: mouse
bool IsMouseButtonPressed(int button);                  // Check if a mouse button has been pressed once
bool IsMouseButtonDown(int button);                     // Check if a mouse button is being pressed
bool IsMouseButtonReleased(int button);                 // Check if a mouse button has been released once
```

```
bool IsMouseButtonUp(int button);                       // Check if a mouse button is NOT being pressed
int GetMouseX(void);                                    // Get mouse position X
int GetMouseY(void);                                    // Get mouse position Y
Vector2 GetMousePosition(void);                         // Get mouse position XY
Vector2 GetMouseDelta(void);                            // Get mouse delta between frames
void SetMousePosition(int x, int y);                    // Set mouse position XY
void SetMouseOffset(int offsetX, int offsetY);          // Set mouse offset
void SetMouseScale(float scaleX, float scaleY);         // Set mouse scaling
float GetMouseWheelMove(void);                          // Get mouse wheel movement for X or Y, whichever is larger
Vector2 GetMouseWheelMoveV(void);                       // Get mouse wheel movement for both X and Y
void SetMouseCursor(int cursor);                        // Set mouse cursor

// Input-related functions: touch
int GetTouchX(void);                                    // Get touch position X for touch point 0 (relative to screen size)
int GetTouchY(void);                                    // Get touch position Y for touch point 0 (relative to screen size)
Vector2 GetTouchPosition(int index);                    // Get touch position XY for a touch point index (relative to screen size)
int GetTouchPointId(int index);                         // Get touch point identifier for given index
int GetTouchPointCount(void);                           // Get number of touch points

//------------------------------------------------------------------------------------
// Gestures and Touch Handling Functions (Module: rgestures)
//------------------------------------------------------------------------------------
void SetGesturesEnabled(unsigned int flags);            // Enable a set of gestures using flags
bool IsGestureDetected(unsigned int gesture);           // Check if a gesture have been detected
int GetGestureDetected(void);                           // Get latest detected gesture
float GetGestureHoldDuration(void);                     // Get gesture hold time in seconds
Vector2 GetGestureDragVector(void);                     // Get gesture drag vector
float GetGestureDragAngle(void);                        // Get gesture drag angle
Vector2 GetGesturePinchVector(void);                    // Get gesture pinch delta
float GetGesturePinchAngle(void);                       // Get gesture pinch angle

//------------------------------------------------------------------------------------
// Camera System Functions (Module: rcamera)
//------------------------------------------------------------------------------------
void UpdateCamera(Camera *camera, int mode);            // Update camera position for selected mode
void UpdateCameraPro(Camera *camera, Vector3 movement, Vector3 rotation, float zoom); // Update camera movement/rotation
```

module: rshapes →

```
// NOTE: It can be useful when using basic shapes and one single font,
// defining a font char white rectangle would allow drawing everything in a single draw call
void SetShapesTexture(Texture2D texture, Rectangle source); // Set texture and rectangle to be used on shapes drawing
Texture2D GetShapesTexture(void);                   // Get texture that is used for shapes drawing
Rectangle GetShapesTextureRectangle(void);          // Get texture source rectangle that is used for shapes drawing

// Basic shapes drawing functions
void DrawPixel(int posX, int posY, Color color);                                                    // Draw a pixel using geometry [Can be slow, use with care]
void DrawPixelV(Vector2 position, Color color);                                                     // Draw a pixel using geometry (Vector version) [Can be slow, use with care]
void DrawLine(int startPosX, int startPosY, int endPosX, int endPosY, Color color);                 // Draw a line
void DrawLineV(Vector2 startPos, Vector2 endPos, Color color);                                      // Draw a line (using gl lines)
void DrawLineEx(Vector2 startPos, Vector2 endPos, float thick, Color color);                        // Draw a line (using triangles/quads)
void DrawLineStrip(const Vector2 *points, int pointCount, Color color);                             // Draw lines sequence (using gl lines)
void DrawLineBezier(Vector2 startPos, Vector2 endPos, float thick, Color color);                    // Draw line segment cubic-bezier in-out interpolation
void DrawLineDashed(Vector2 startPos, Vector2 endPos, int dashSize, int spaceSize, Color color);    // Draw a dashed line
void DrawCircle(int centerX, int centerY, float radius, Color color);                               // Draw a color-filled circle
void DrawCircleV(Vector2 center, float radius, Color color);                                        // Draw a color-filled circle (Vector version)
void DrawCircleGradient(Vector2 center, float radius, Color inner, Color outer);                    // Draw a gradient-filled circle
void DrawCircleSector(Vector2 center, float radius, float startAngle, float endAngle, int segments, Color color); // Draw a piece of a circle
void DrawCircleSectorLines(Vector2 center, float radius, float startAngle, float endAngle, int segments, Color color); // Draw circle sector outline
void DrawCircleLines(int centerX, int centerY, float radius, Color color);                          // Draw circle outline
```

```
void DrawCircleLinesV(Vector2 center, float radius, Color color);                                 // Draw circle outline (Vector version)
void DrawEllipse(int centerX, int centerY, float radiusH, float radiusV, Color color);            // Draw ellipse
void DrawEllipseV(Vector2 center, float radiusH, float radiusV, Color color);                     // Draw ellipse (Vector version)
void DrawEllipseLines(int centerX, int centerY, float radiusH, float radiusV, Color color);       // Draw ellipse outline
void DrawEllipseLinesV(Vector2 center, float radiusH, float radiusV, Color color);                // Draw ellipse outline (Vector version)
void DrawRing(Vector2 center, float innerRadius, float outerRadius, float startAngle, float endAngle, int segments, Color color); // Draw ring
void DrawRingLines(Vector2 center, float innerRadius, float outerRadius, float startAngle, float endAngle, int segments, Color color); // Draw ring outline
void DrawRectangle(int posX, int posY, int width, int height, Color color);                       // Draw a color-filled rectangle
void DrawRectangleV(Vector2 position, Vector2 size, Color color);                                 // Draw a color-filled rectangle (Vector version)
void DrawRectangleRec(Rectangle rec, Color color);                                                // Draw a color-filled rectangle
void DrawRectanglePro(Rectangle rec, Vector2 origin, float rotation, Color color);                // Draw a color-filled rectangle with pro parameters
void DrawRectangleGradientV(int posX, int posY, int width, int height, Color top, Color bottom);  // Draw a vertical-gradient-filled rectangle
void DrawRectangleGradientH(int posX, int posY, int width, int height, Color left, Color right);  // Draw a horizontal-gradient-filled rectangle
void DrawRectangleGradientEx(Rectangle rec, Color topLeft, Color bottomLeft, Color bottomRight, Color topRight); // Draw a gradient-filled rectangle with custom vertex colors
void DrawRectangleLines(int posX, int posY, int width, int height, Color color);                  // Draw rectangle outline
void DrawRectangleLinesEx(Rectangle rec, float lineThick, Color color);                           // Draw rectangle outline with extended parameters
void DrawRectangleRounded(Rectangle rec, float roundness, int segments, Color color);             // Draw rectangle with rounded edges
void DrawRectangleRoundedLines(Rectangle rec, float roundness, int segments, Color color);        // Draw rectangle lines with rounded edges
void DrawRectangleRoundedLinesEx(Rectangle rec, float roundness, int segments, float lineThick, Color color); // Draw rectangle with rounded edges outline
void DrawTriangle(Vector2 v1, Vector2 v2, Vector2 v3, Color color);                               // Draw a color-filled triangle (vertex in counter-clockwise order!)
void DrawTriangleLines(Vector2 v1, Vector2 v2, Vector2 v3, Color color);                          // Draw triangle outline (vertex in counter-clockwise order!)
void DrawTriangleFan(const Vector2 *points, int pointCount, Color color);                         // Draw a triangle fan defined by points (first vertex is the center)
void DrawTriangleStrip(const Vector2 *points, int pointCount, Color color);                       // Draw a triangle strip defined by points
void DrawPoly(Vector2 center, int sides, float radius, float rotation, Color color);              // Draw a regular polygon (Vector version)
void DrawPolyLines(Vector2 center, int sides, float radius, float rotation, Color color);         // Draw a polygon outline of n sides
void DrawPolyLinesEx(Vector2 center, int sides, float radius, float rotation, float lineThick, Color color); // Draw a polygon outline of n sides with extended parameters

// Splines drawing functions
void DrawSplineLinear(const Vector2 *points, int pointCount, float thick, Color color);          // Draw spline: Linear, minimum 2 points
void DrawSplineBasis(const Vector2 *points, int pointCount, float thick, Color color);           // Draw spline: B-Spline, minimum 4 points
void DrawSplineCatmullRom(const Vector2 *points, int pointCount, float thick, Color color);      // Draw spline: Catmull-Rom, minimum 4 points
void DrawSplineBezierQuadratic(const Vector2 *points, int pointCount, float thick, Color color); // Draw spline: Quadratic Bezier, minimum 3 points (1 control point): [p1, c2, p3, c4...]
void DrawSplineBezierCubic(const Vector2 *points, int pointCount, float thick, Color color);     // Draw spline: Cubic Bezier, minimum 4 points (2 control points): [p1, c2, c3, p4, c5, c6...]
void DrawSplineSegmentLinear(Vector2 p1, Vector2 p2, float thick, Color color);                   // Draw spline segment: Linear, 2 points
void DrawSplineSegmentBasis(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float thick, Color color); // Draw spline segment: B-Spline, 4 points
void DrawSplineSegmentCatmullRom(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float thick, Color color); // Draw spline segment: Catmull-Rom, 4 points
void DrawSplineSegmentBezierQuadratic(Vector2 p1, Vector2 c2, Vector2 p3, float thick, Color color); // Draw spline segment: Quadratic Bezier, 2 points, 1 control point
void DrawSplineSegmentBezierCubic(Vector2 p1, Vector2 c2, Vector2 c3, Vector2 p4, float thick, Color color); // Draw spline segment: Cubic Bezier, 2 points, 2 control points

// Spline segment point evaluation functions, for a given t [0.0f .. 1.0f]
Vector2 GetSplinePointLinear(Vector2 startPos, Vector2 endPos, float t);                          // Get (evaluate) spline point: Linear
Vector2 GetSplinePointBasis(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float t);             // Get (evaluate) spline point: B-Spline
Vector2 GetSplinePointCatmullRom(Vector2 p1, Vector2 p2, Vector2 p3, Vector2 p4, float t);        // Get (evaluate) spline point: Catmull-Rom
Vector2 GetSplinePointBezierQuad(Vector2 p1, Vector2 c2, Vector2 p3, float t);                    // Get (evaluate) spline point: Quadratic Bezier
Vector2 GetSplinePointBezierCubic(Vector2 p1, Vector2 c2, Vector2 c3, Vector2 p4, float t);       // Get (evaluate) spline point: Cubic Bezier

// Basic shapes collision detection functions
bool CheckCollisionRecs(Rectangle rec1, Rectangle rec2);                                          // Check collision between two rectangles
bool CheckCollisionCircles(Vector2 center1, float radius1, Vector2 center2, float radius2);       // Check collision between two circles
bool CheckCollisionCircleRec(Vector2 center, float radius, Rectangle rec);                        // Check collision between circle and rectangle
bool CheckCollisionCircleLine(Vector2 center, float radius, Vector2 p1, Vector2 p2);              // Check if circle collides with a line created betweeen two points [p1] and [p2]
bool CheckCollisionPointRec(Vector2 point, Rectangle rec);                                        // Check if point is inside rectangle
bool CheckCollisionPointCircle(Vector2 point, Vector2 center, float radius);                      // Check if point is inside circle
bool CheckCollisionPointTriangle(Vector2 point, Vector2 p1, Vector2 p2, Vector2 p3);              // Check if point is inside a triangle
bool CheckCollisionPointLine(Vector2 point, Vector2 p1, Vector2 p2, int threshold);               // Check if point belongs to line created between two points [p1] and [p2] with defined margin in
bool CheckCollisionPointPoly(Vector2 point, const Vector2 *points, int pointCount);               // Check if point is within a polygon described by array of vertices
bool CheckCollisionLines(Vector2 startPos1, Vector2 endPos1, Vector2 startPos2, Vector2 endPos2, Vector2 *collisionPoint); // Check the collision between two lines defined by two points each, retur
Rectangle GetCollisionRec(Rectangle rec1, Rectangle rec2);                                        // Get collision rectangle for two rectangles collision
```

```
// Image loading functions
// NOTE: These functions do not require GPU access
Image LoadImage(const char *fileName);                                                                  // Load image from file into CPU memory (RAM)
Image LoadImageRaw(const char *fileName, int width, int height, int format, int headerSize);           // Load image from RAW file data
Image LoadImageAnim(const char *fileName, int *frames);                                                 // Load image sequence from file (frames appended to image.data)
Image LoadImageAnimFromMemory(const char *fileType, const unsigned char *fileData, int dataSize, int *frames); // Load image sequence from memory buffer
Image LoadImageFromMemory(const char *fileType, const unsigned char *fileData, int dataSize);          // Load image from memory buffer, fileType refers to extension: i.e. '.png'
Image LoadImageFromTexture(Texture2D texture);                                                          // Load image from GPU texture data
Image LoadImageFromScreen(void);                                                                        // Load image from screen buffer and (screenshot)
bool IsImageValid(Image image);                                                                         // Check if an image is valid (data and parameters)
void UnloadImage(Image image);                                                                          // Unload image from CPU memory (RAM)
bool ExportImage(Image image, const char *fileName);                                                    // Export image data to file, returns true on success
unsigned char *ExportImageToMemory(Image image, const char *fileType, int *fileSize);                   // Export image to memory buffer, memory must be MemFree()
bool ExportImageAsCode(Image image, const char *fileName);                                              // Export image as code file defining an array of bytes, returns true on success

// Image generation functions
Image GenImageColor(int width, int height, Color color);                                                // Generate image: plain color
Image GenImageGradientLinear(int width, int height, int direction, Color start, Color end);            // Generate image: linear gradient, direction in degrees [0..360], 0=Vertical gradient
Image GenImageGradientRadial(int width, int height, float density, Color inner, Color outer);          // Generate image: radial gradient
Image GenImageGradientSquare(int width, int height, float density, Color inner, Color outer);          // Generate image: square gradient
Image GenImageChecked(int width, int height, int checksX, int checksY, Color col1, Color col2);        // Generate image: checked
Image GenImageWhiteNoise(int width, int height, float factor);                                          // Generate image: white noise
Image GenImagePerlinNoise(int width, int height, int offsetX, int offsetY, float scale);                // Generate image: perlin noise
Image GenImageCellular(int width, int height, int tileSize);                                            // Generate image: cellular algorithm, bigger tileSize means bigger cells
Image GenImageText(int width, int height, const char *text);                                            // Generate image: grayscale image from text data

// Image manipulation functions
Image ImageCopy(Image image);                                                                           // Create an image duplicate (useful for transformations)
Image ImageFromImage(Image image, Rectangle rec);                                                       // Create an image from another image piece
Image ImageFromChannel(Image image, int selectedChannel);                                               // Create an image from a selected channel of another image (GRAYSCALE)
Image ImageText(const char *text, int fontSize, Color color);                                           // Create an image from text (default font)
Image ImageTextEx(Font font, const char *text, float fontSize, float spacing, Color tint);              // Create an image from text (custom sprite font)
void ImageFormat(Image *image, int newFormat);                                                          // Convert image data to desired format
void ImageToPOT(Image *image, Color fill);                                                              // Convert image to POT (power-of-two)
void ImageCrop(Image *image, Rectangle crop);                                                           // Crop an image to a defined rectangle
void ImageAlphaCrop(Image *image, float threshold);                                                     // Crop image depending on alpha value
void ImageAlphaClear(Image *image, Color color, float threshold);                                       // Clear alpha channel to desired color
void ImageAlphaMask(Image *image, Image alphaMask);                                                     // Apply alpha mask to image
void ImageAlphaPremultiply(Image *image);                                                               // Premultiply alpha channel
void ImageBlurGaussian(Image *image, int blurSize);                                                     // Apply Gaussian blur using a box blur approximation
void ImageKernelConvolution(Image *image, const float *kernel, int kernelSize);                         // Apply custom square convolution kernel to image
void ImageResize(Image *image, int newWidth, int newHeight);                                            // Resize image (Bicubic scaling algorithm)
void ImageResizeNN(Image *image, int newWidth, int newHeight);                                          // Resize image (Nearest-Neighbor scaling algorithm)
void ImageResizeCanvas(Image *image, int newWidth, int newHeight, int offsetX, int offsetY, Color fill); // Resize canvas and fill with color
void ImageMipmaps(Image *image);                                                                        // Compute all mipmap levels for a provided image
void ImageDither(Image *image, int rBpp, int gBpp, int bBpp, int aBpp);                                 // Dither image data to 16bpp or lower (Floyd-Steinberg dithering)
void ImageFlipVertical(Image *image);                                                                   // Flip image vertically
void ImageFlipHorizontal(Image *image);                                                                 // Flip image horizontally
void ImageRotate(Image *image, int degrees);                                                            // Rotate image by input angle in degrees (-359 to 359)
void ImageRotateCW(Image *image);                                                                       // Rotate image clockwise 90deg
void ImageRotateCCW(Image *image);                                                                      // Rotate image counter-clockwise 90deg
void ImageColorTint(Image *image, Color color);                                                         // Modify image color: tint
void ImageColorInvert(Image *image);                                                                    // Modify image color: invert
void ImageColorGrayscale(Image *image);                                                                 // Modify image color: grayscale
void ImageColorContrast(Image *image, float contrast);                                                  // Modify image color: contrast (-100 to 100)
void ImageColorBrightness(Image *image, int brightness);                                                // Modify image color: brightness (-255 to 255)
void ImageColorReplace(Image *image, Color color, Color replace);                                       // Modify image color: replace color
Color *LoadImageColors(Image image);                                                                    // Load color data from image as a Color array (RGBA - 32bit)
Color *LoadImagePalette(Image image, int maxPaletteSize, int *colorCount);                              // Load colors palette from image as a Color array (RGBA - 32bit)
void UnloadImageColors(Color *colors);                                                                  // Unload color data loaded with LoadImageColors()
void UnloadImagePalette(Color *colors);                                                                 // Unload colors palette loaded with LoadImagePalette()
Rectangle GetImageAlphaBorder(Image image, float threshold);                                            // Get image alpha border rectangle
Color GetImageColor(Image image, int x, int y);                                                         // Get image pixel color at (x, y) position
```

```
// Image drawing functions
// NOTE: Image software-rendering functions (CPU)
void ImageClearBackground(Image *dst, Color color);                                                 // Clear image background with given color
void ImageDrawPixel(Image *dst, int posX, int posY, Color color);                                   // Draw pixel within an image
void ImageDrawPixelV(Image *dst, Vector2 position, Color color);                                    // Draw pixel within an image (Vector version)
void ImageDrawLine(Image *dst, int startPosX, int startPosY, int endPosX, int endPosY, Color color); // Draw line within an image
void ImageDrawLineV(Image *dst, Vector2 start, Vector2 end, Color color);                           // Draw line within an image (Vector version)
void ImageDrawLineEx(Image *dst, Vector2 start, Vector2 end, int thick, Color color);               // Draw a line defining thickness within an image
void ImageDrawCircle(Image *dst, int centerX, int centerY, int radius, Color color);                // Draw a filled circle within an image
void ImageDrawCircleV(Image *dst, Vector2 center, int radius, Color color);                         // Draw a filled circle within an image (Vector version)
void ImageDrawCircleLines(Image *dst, int centerX, int centerY, int radius, Color color);           // Draw circle outline within an image
void ImageDrawCircleLinesV(Image *dst, Vector2 center, int radius, Color color);                    // Draw circle outline within an image (Vector version)
void ImageDrawRectangle(Image *dst, int posX, int posY, int width, int height, Color color);       // Draw rectangle within an image
void ImageDrawRectangleV(Image *dst, Vector2 position, Vector2 size, Color color);                  // Draw rectangle within an image (Vector version)
void ImageDrawRectangleRec(Image *dst, Rectangle rec, Color color);                                 // Draw rectangle within an image
void ImageDrawRectangleLines(Image *dst, Rectangle rec, int thick, Color color);                    // Draw rectangle lines within an image
void ImageDrawTriangle(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);                // Draw triangle within an image
void ImageDrawTriangleEx(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color c1, Color c2, Color c3); // Draw triangle with interpolated colors within an image
void ImageDrawTriangleLines(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);           // Draw triangle outline within an image
void ImageDrawTriangleFan(Image *dst, const Vector2 *points, int pointCount, Color color);         // Draw a triangle fan defined by points within an image (first vertex is the center)
void ImageDrawTriangleStrip(Image *dst, const Vector2 *points, int pointCount, Color color);       // Draw a triangle strip defined by points within an image
void ImageDraw(Image *dst, Image src, Rectangle srcRec, Rectangle dstRec, Color tint);              // Draw a source image within a destination image (tint applied to source)
void ImageDrawText(Image *dst, const char *text, int posX, int posY, int fontSize, Color color);   // Draw text (using default font) within an image (destination)
void ImageDrawTextEx(Image *dst, Font font, const char *text, Vector2 position, float fontSize, float spacing, Color tint); // Draw text (custom sprite font) within an image (destination)

// Texture loading functions
// NOTE: These functions require GPU access
Texture2D LoadTexture(const char *fileName);                                                        // Load texture from file into GPU memory (VRAM)
Texture2D LoadTextureFromImage(Image image);                                                        // Load texture from image data
TextureCubemap LoadTextureCubemap(Image image, int layout);                                         // Load cubemap from image, multiple image cubemap layouts supported
RenderTexture2D LoadRenderTexture(int width, int height);                                           // Load texture for rendering (framebuffer)
bool IsTextureValid(Texture2D texture);                                                             // Check if a texture is valid (loaded in GPU)
void UnloadTexture(Texture2D texture);                                                              // Unload texture from GPU memory (VRAM)
bool IsRenderTextureValid(RenderTexture2D target);                                                  // Check if a render texture is valid (loaded in GPU)
void UnloadRenderTexture(RenderTexture2D target);                                                   // Unload render texture from GPU memory (VRAM)
void UpdateTexture(Texture2D texture, const void *pixels);                                          // Update GPU texture with new data (pixels should be able to fill texture)
void UpdateTextureRec(Texture2D texture, Rectangle rec, const void *pixels);                        // Update GPU texture rectangle with new data (pixels and rec should fit in texture)

// Texture configuration functions
void GenTextureMipmaps(Texture2D *texture);                                                         // Generate GPU mipmaps for a texture
void SetTextureFilter(Texture2D texture, int filter);                                               // Set texture scaling filter mode
void SetTextureWrap(Texture2D texture, int wrap);                                                   // Set texture wrapping mode

// Texture drawing functions
void DrawTexture(Texture2D texture, int posX, int posY, Color tint);                                // Draw a Texture2D
void DrawTextureV(Texture2D texture, Vector2 position, Color tint);                                 // Draw a Texture2D with position defined as Vector2
void DrawTextureEx(Texture2D texture, Vector2 position, float rotation, float scale, Color tint);   // Draw a Texture2D with extended parameters
void DrawTextureRec(Texture2D texture, Rectangle source, Vector2 position, Color tint);             // Draw a part of a texture defined by a rectangle
void DrawTexturePro(Texture2D texture, Rectangle source, Rectangle dest, Vector2 origin, float rotation, Color tint); // Draw a part of a texture defined by a rectangle with 'pro' parameters
void DrawTextureNPatch(Texture2D texture, NPatchInfo nPatchInfo, Rectangle dest, Vector2 origin, float rotation, Color tint); // Draws a texture (or part of it) that stretches or shrinks nicely

// Color/pixel related functions
bool ColorIsEqual(Color col1, Color col2);                          // Check if two colors are equal
Color Fade(Color color, float alpha);                               // Get color with alpha applied, alpha goes from 0.0f to 1.0f
int ColorToInt(Color color);                                        // Get hexadecimal value for a Color (0xRRGGBBAA)
Vector4 ColorNormalize(Color color);                                // Get Color normalized as float [0..1]
Color ColorFromNormalized(Vector4 normalized);                      // Get Color from normalized values [0..1]
Vector3 ColorToHSV(Color color);                                    // Get HSV values for a Color, hue [0..360], saturation/value [0..1]
Color ColorFromHSV(float hue, float saturation, float value);       // Get a Color from HSV values, hue [0..360], saturation/value [0..1]
Color ColorTint(Color color, Color tint);                           // Get color multiplied with another color
Color ColorBrightness(Color color, float factor);                   // Get color with brightness correction, brightness factor goes from -1.0f to 1.0f
Color ColorContrast(Color color, float contrast);                   // Get color with contrast correction, contrast values between -1.0f and 1.0f
Color ColorAlpha(Color color, float alpha);                         // Get color with alpha applied, alpha goes from 0.0f to 1.0f
```

```
Color ColorAlphaBlend(Color dst, Color src, Color tint);          // Get src alpha-blended into dst color with tint
Color ColorLerp(Color color1, Color color2, float factor);        // Get color lerp interpolation between two colors, factor [0.0f..1.0f]
Color GetColor(unsigned int hexValue);                            // Get Color structure from hexadecimal value
Color GetPixelColor(void *srcPtr, int format);                    // Get Color from a source pixel pointer of certain format
void SetPixelColor(void *dstPtr, Color color, int format);        // Set color formatted into destination pixel pointer
int GetPixelDataSize(int width, int height, int format);          // Get pixel data size in bytes for certain format
```

module: rtext →

```
// Font loading/unloading functions
Font GetFontDefault(void);                                                                  // Get the default Font
Font LoadFont(const char *fileName);                                                        // Load font from file into GPU memory (VRAM)
Font LoadFontEx(const char *fileName, int fontSize, const int *codepoints, int codepointCount); // Load font from file with extended parameters, use NULL for codepoints and 0 for codepointCount to
Font LoadFontFromImage(Image image, Color key, int firstChar);                              // Load font from Image (XNA style)
Font LoadFontFromMemory(const char *fileType, const unsigned char *fileData, int dataSize, int fontSize, const int *codepoints, int codepointCount); // Load font from memory buffer, fileType refers
bool IsFontValid(Font font);                                                                // Check if a font is valid (font data loaded, WARNING: GPU texture not checked)
GlyphInfo *LoadFontData(const unsigned char *fileData, int dataSize, int fontSize, const int *codepoints, int codepointCount, int type, int *glyphCount); // Load font data for further use
Image GenImageFontAtlas(const GlyphInfo *glyphs, Rectangle **glyphRecs, int glyphCount, int fontSize, int padding, int packMethod); // Generate image font atlas using chars info
void UnloadFontData(GlyphInfo *glyphs, int glyphCount);                                     // Unload font chars info data (RAM)
void UnloadFont(Font font);                                                                 // Unload font from GPU memory (VRAM)
bool ExportFontAsCode(Font font, const char *fileName);                                     // Export font as code file, returns true on success

// Text drawing functions
void DrawFPS(int posX, int posY);                                                           // Draw current FPS
void DrawText(const char *text, int posX, int posY, int fontSize, Color color);             // Draw text (using default font)
void DrawTextEx(Font font, const char *text, Vector2 position, float fontSize, float spacing, Color tint); // Draw text using font and additional parameters
void DrawTextPro(Font font, const char *text, Vector2 position, Vector2 origin, float rotation, float fontSize, float spacing, Color tint); // Draw text using Font and pro parameters (rotation)
void DrawTextCodepoint(Font font, int codepoint, Vector2 position, float fontSize, Color tint); // Draw one character (codepoint)
void DrawTextCodepoints(Font font, const int *codepoints, int codepointCount, Vector2 position, float fontSize, float spacing, Color tint); // Draw multiple character (codepoint)

// Text font info functions
void SetTextLineSpacing(int spacing);                                                       // Set vertical line spacing when drawing with line-breaks
int MeasureText(const char *text, int fontSize);                                            // Measure string width for default font
Vector2 MeasureTextEx(Font font, const char *text, float fontSize, float spacing);          // Measure string size for Font
Vector2 MeasureTextCodepoints(Font font, const int *codepoints, int length, float fontSize, float spacing); // Measure string size for an existing array of codepoints for Font
int GetGlyphIndex(Font font, int codepoint);                                                // Get glyph index position in font for a codepoint (unicode character), fallback to '?' if not found
GlyphInfo GetGlyphInfo(Font font, int codepoint);                                           // Get glyph font info data for a codepoint (unicode character), fallback to '?' if not found
Rectangle GetGlyphAtlasRec(Font font, int codepoint);                                       // Get glyph rectangle in font atlas for a codepoint (unicode character), fallback to '?' if not found

// Text codepoints management functions (unicode characters)
char *LoadUTF8(const int *codepoints, int length);                                          // Load UTF-8 text encoded from codepoints array
void UnloadUTF8(char *text);                                                                // Unload UTF-8 text encoded from codepoints array
int *LoadCodepoints(const char *text, int *count);                                          // Load all codepoints from a UTF-8 text string, codepoints count returned by parameter
void UnloadCodepoints(int *codepoints);                                                     // Unload codepoints data from memory
int GetCodepointCount(const char *text);                                                    // Get total number of codepoints in a UTF-8 encoded string
int GetCodepoint(const char *text, int *codepointSize);                                     // Get next codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
int GetCodepointNext(const char *text, int *codepointSize);                                 // Get next codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
int GetCodepointPrevious(const char *text, int *codepointSize);                             // Get previous codepoint in a UTF-8 encoded string, 0x3f('?') is returned on failure
const char *CodepointToUTF8(int codepoint, int *utf8Size);                                  // Encode one codepoint into UTF-8 byte array (array length returned as parameter)

// Text strings management functions (no UTF-8 strings, only byte chars)
// WARNING 1: Most of these functions use internal static buffers[], it's recommended to store returned data on user-side for re-use
// WARNING 2: Some functions allocate memory internally for the returned strings, those strings must be freed by user using MemFree()
char **LoadTextLines(const char *text, int *count);                                         // Load text as separate lines ('\n')
void UnloadTextLines(char **text, int lineCount);                                           // Unload text lines
int TextCopy(char *dst, const char *src);                                                   // Copy one string to another, returns bytes copied
bool TextIsEqual(const char *text1, const char *text2);                                     // Check if two text string are equal
unsigned int TextLength(const char *text);                                                  // Get text length, checks for '\0' ending
const char *TextFormat(const char *text, ...);                                              // Text formatting with variables (sprintf() style)
const char *TextSubtext(const char *text, int position, int length);                        // Get a piece of a text string
```

```
const char *TextRemoveSpaces(const char *text);                                     // Remove text spaces, concat words
char *GetTextBetween(const char *text, const char *begin, const char *end);         // Get text between two strings
char *TextReplace(const char *text, const char *search, const char *replacement);   // Replace text string with new string
char *TextReplaceAlloc(const char *text, const char *search, const char *replacement); // Replace text string with new string, memory must be MemFree()
char *TextReplaceBetween(const char *text, const char *begin, const char *end, const char *replacement); // Replace text between two specific strings
char *TextReplaceBetweenAlloc(const char *text, const char *begin, const char *end, const char *replacement); // Replace text between two specific strings, memory must be MemFree()
char *TextInsert(const char *text, const char *insert, int position);               // Insert text in a defined byte position
char *TextInsertAlloc(const char *text, const char *insert, int position);          // Insert text in a defined byte position, memory must be MemFree()
char *TextJoin(char **textList, int count, const char *delimiter);                  // Join text strings with delimiter
char **TextSplit(const char *text, char delimiter, int *count);                     // Split text into multiple strings, using MAX_TEXTSPLIT_COUNT static strings
void TextAppend(char *text, const char *append, int *position);                     // Append text at specific position and move cursor
int TextFindIndex(const char *text, const char *search);                            // Find first text occurrence within a string, -1 if not found
char *TextToUpper(const char *text);                                                // Get upper case version of provided string
char *TextToLower(const char *text);                                                // Get lower case version of provided string
char *TextToPascal(const char *text);                                               // Get Pascal case notation version of provided string
char *TextToSnake(const char *text);                                                // Get Snake case notation version of provided string
char *TextToCamel(const char *text);                                                // Get Camel case notation version of provided string
int TextToInteger(const char *text);                                                // Get integer value from text
float TextToFloat(const char *text);                                                // Get float value from text
```

module: rmodels →

```
// Basic geometric 3D shapes drawing functions
void DrawLine3D(Vector3 startPos, Vector3 endPos, Color color);                                         // Draw a line in 3D world space
void DrawPoint3D(Vector3 position, Color color);                                                        // Draw a point in 3D space, actually a small line
void DrawCircle3D(Vector3 center, float radius, Vector3 rotationAxis, float rotationAngle, Color color); // Draw a circle in 3D world space
void DrawTriangle3D(Vector3 v1, Vector3 v2, Vector3 v3, Color color);                                   // Draw a color-filled triangle (vertex in counter-clockwise order!)
void DrawTriangleStrip3D(const Vector3 *points, int pointCount, Color color);                          // Draw a triangle strip defined by points
void DrawCube(Vector3 position, float width, float height, float length, Color color);                 // Draw cube
void DrawCubeV(Vector3 position, Vector3 size, Color color);                                           // Draw cube (Vector version)
void DrawCubeWires(Vector3 position, float width, float height, float length, Color color);            // Draw cube wires
void DrawCubeWiresV(Vector3 position, Vector3 size, Color color);                                      // Draw cube wires (Vector version)
void DrawSphere(Vector3 centerPos, float radius, Color color);                                         // Draw sphere
void DrawSphereEx(Vector3 centerPos, float radius, int rings, int slices, Color color);                // Draw sphere with extended parameters
void DrawSphereWires(Vector3 centerPos, float radius, int rings, int slices, Color color);             // Draw sphere wires
void DrawCylinder(Vector3 position, float radiusTop, float radiusBottom, float height, int slices, Color color); // Draw a cylinder/cone
void DrawCylinderEx(Vector3 startPos, Vector3 endPos, float startRadius, float endRadius, int sides, Color color); // Draw a cylinder with base at startPos and top at endPos
void DrawCylinderWires(Vector3 position, float radiusTop, float radiusBottom, float height, int slices, Color color); // Draw a cylinder/cone wires
void DrawCylinderWiresEx(Vector3 startPos, Vector3 endPos, float startRadius, float endRadius, int sides, Color color); // Draw a cylinder wires with base at startPos and top at endPos
void DrawCapsule(Vector3 startPos, Vector3 endPos, float radius, int slices, int rings, Color color); // Draw a capsule with the center of its sphere caps at startPos and endPos
void DrawCapsuleWires(Vector3 startPos, Vector3 endPos, float radius, int slices, int rings, Color color); // Draw capsule wireframe with the center of its sphere caps at startPos and endPos
void DrawPlane(Vector3 centerPos, Vector2 size, Color color);                                          // Draw a plane XZ
void DrawRay(Ray ray, Color color);                                                                    // Draw a ray line
void DrawGrid(int slices, float spacing);                                                              // Draw a grid (centered at (0, 0, 0))

//------------------------------------------------------------------------------------
// Model 3d Loading and Drawing Functions (Module: models)
//------------------------------------------------------------------------------------

// Model management functions
Model LoadModel(const char *fileName);                                                  // Load model from files (meshes and materials)
Model LoadModelFromMesh(Mesh mesh);                                                     // Load model from generated mesh (default material)
bool IsModelValid(Model model);                                                         // Check if a model is valid (loaded in GPU, VAO/VBOs)
void UnloadModel(Model model);                                                          // Unload model (including meshes) from memory (RAM and/or VRAM)
BoundingBox GetModelBoundingBox(Model model);                                           // Compute model bounding box limits (considers all meshes)

// Model drawing functions
void DrawModel(Model model, Vector3 position, float scale, Color tint);                 // Draw a model (with texture if set)
void DrawModelEx(Model model, Vector3 position, Vector3 rotationAxis, float rotationAngle, Vector3 scale, Color tint); // Draw a model with extended parameters
void DrawModelWires(Model model, Vector3 position, float scale, Color tint);            // Draw a model wires (with texture if set)
```

```
void DrawModelWiresEx(Model model, Vector3 position, Vector3 rotationAxis, float rotationAngle, Vector3 scale, Color tint); // Draw a model wires (with texture if set) with extended parameters
void DrawBoundingBox(BoundingBox box, Color color);                                                                         // Draw bounding box (wires)
void DrawBillboard(Camera camera, Texture2D texture, Vector3 position, float scale, Color tint); // Draw a billboard texture
void DrawBillboardRec(Camera camera, Texture2D texture, Rectangle source, Vector3 position, Vector2 size, Color tint); // Draw a billboard texture defined by source
void DrawBillboardPro(Camera camera, Texture2D texture, Rectangle source, Vector3 position, Vector3 up, Vector2 size, Vector2 origin, float rotation, Color tint); // Draw a billboard texture define

// Mesh management functions
void UploadMesh(Mesh *mesh, bool dynamic);                                                  // Upload mesh vertex data in GPU and provide VAO/VBO ids
void UpdateMeshBuffer(Mesh mesh, int index, const void *data, int dataSize, int offset); // Update mesh vertex data in GPU for a specific buffer index
void UnloadMesh(Mesh mesh);                                                                 // Unload mesh data from CPU and GPU
void DrawMesh(Mesh mesh, Material material, Matrix transform);                              // Draw a 3d mesh with material and transform
void DrawMeshInstanced(Mesh mesh, Material material, const Matrix *transforms, int instances); // Draw multiple mesh instances with material and different transforms
BoundingBox GetMeshBoundingBox(Mesh mesh);                                                  // Compute mesh bounding box limits
void GenMeshTangents(Mesh *mesh);                                                           // Compute mesh tangents
bool ExportMesh(Mesh mesh, const char *fileName);                                           // Export mesh data to file, returns true on success
bool ExportMeshAsCode(Mesh mesh, const char *fileName);                                     // Export mesh as code file (.h) defining multiple arrays of vertex attributes

// Mesh generation functions
Mesh GenMeshPoly(int sides, float radius);                                                  // Generate polygonal mesh
Mesh GenMeshPlane(float width, float length, int resX, int resZ);                           // Generate plane mesh (with subdivisions)
Mesh GenMeshCube(float width, float height, float length);                                  // Generate cuboid mesh
Mesh GenMeshSphere(float radius, int rings, int slices);                                    // Generate sphere mesh (standard sphere)
Mesh GenMeshHemiSphere(float radius, int rings, int slices);                                // Generate half-sphere mesh (no bottom cap)
Mesh GenMeshCylinder(float radius, float height, int slices);                               // Generate cylinder mesh
Mesh GenMeshCone(float radius, float height, int slices);                                   // Generate cone/pyramid mesh
Mesh GenMeshTorus(float radius, float size, int radSeg, int sides);                         // Generate torus mesh
Mesh GenMeshKnot(float radius, float size, int radSeg, int sides);                          // Generate trefoil knot mesh
Mesh GenMeshHeightmap(Image heightmap, Vector3 size);                                       // Generate heightmap mesh from image data
Mesh GenMeshCubicmap(Image cubicmap, Vector3 cubeSize);                                     // Generate cubes-based map mesh from image data

// Material loading/unloading functions
Material *LoadMaterials(const char *fileName, int *materialCount);                          // Load materials from model file
Material LoadMaterialDefault(void);                                                         // Load default material (Supports: DIFFUSE, SPECULAR, NORMAL maps)
bool IsMaterialValid(Material material);                                                    // Check if a material is valid (shader assigned, map textures loaded in GPU)
void UnloadMaterial(Material material);                                                     // Unload material from GPU memory (VRAM)
void SetMaterialTexture(Material *material, int mapType, Texture2D texture);               // Set texture for a material map type (MATERIAL_MAP_DIFFUSE, MATERIAL_MAP_SPECULAR...)
void SetModelMeshMaterial(Model *model, int meshId, int materialId);                        // Set material for a mesh

// Model animations loading/unloading functions
ModelAnimation *LoadModelAnimations(const char *fileName, int *animCount);                  // Load model animations from file
void UpdateModelAnimation(Model model, ModelAnimation anim, float frame);                   // Update model animation pose (vertex buffers and bone matrices)
void UpdateModelAnimationEx(Model model, ModelAnimation animA, float frameA, ModelAnimation animB, float frameB, float blend); // Update model animation pose, blending two animations
void UnloadModelAnimations(ModelAnimation *animations, int animCount);                      // Unload animation array data
bool IsModelAnimationValid(Model model, ModelAnimation anim);                               // Check model animation skeleton match

// Collision detection functions
bool CheckCollisionSpheres(Vector3 center1, float radius1, Vector3 center2, float radius2); // Check collision between two spheres
bool CheckCollisionBoxes(BoundingBox box1, BoundingBox box2);                               // Check collision between two bounding boxes
bool CheckCollisionBoxSphere(BoundingBox box, Vector3 center, float radius);                // Check collision between box and sphere
RayCollision GetRayCollisionSphere(Ray ray, Vector3 center, float radius);                  // Get collision info between ray and sphere
RayCollision GetRayCollisionBox(Ray ray, BoundingBox box);                                  // Get collision info between ray and box
RayCollision GetRayCollisionMesh(Ray ray, Mesh mesh, Matrix transform);                     // Get collision info between ray and mesh
RayCollision GetRayCollisionTriangle(Ray ray, Vector3 p1, Vector3 p2, Vector3 p3);          // Get collision info between ray and triangle
RayCollision GetRayCollisionQuad(Ray ray, Vector3 p1, Vector3 p2, Vector3 p3, Vector3 p4); // Get collision info between ray and quad
```

module: raudio →

```
// Audio device management functions
void InitAudioDevice(void);                                 // Initialize audio device and context
void CloseAudioDevice(void);                                // Close the audio device and context
```

```
bool IsAudioDeviceReady(void);                              // Check if audio device has been initialized successfully
void SetMasterVolume(float volume);                         // Set master volume (listener)
float GetMasterVolume(void);                                // Get master volume (listener)

// Wave/Sound loading/unloading functions
Wave LoadWave(const char *fileName);                        // Load wave data from file
Wave LoadWaveFromMemory(const char *fileType, const unsigned char *fileData, int dataSize); // Load wave from memory buffer, fileType refers to extension: i.e. '.wav'
bool IsWaveValid(Wave wave);                                // Checks if wave data is valid (data loaded and parameters)
Sound LoadSound(const char *fileName);                      // Load sound from file
Sound LoadSoundFromWave(Wave wave);                         // Load sound from wave data
Sound LoadSoundAlias(Sound source);                         // Create a new sound that shares the same sample data as the source sound, does not own the sound data
bool IsSoundValid(Sound sound);                             // Checks if a sound is valid (data loaded and buffers initialized)
void UpdateSound(Sound sound, const void *data, int sampleCount); // Update sound buffer with new data (default data format: 32 bit float, stereo)
void UnloadWave(Wave wave);                                 // Unload wave data
void UnloadSound(Sound sound);                              // Unload sound
void UnloadSoundAlias(Sound alias);                         // Unload a sound alias (does not deallocate sample data)
bool ExportWave(Wave wave, const char *fileName);           // Export wave data to file, returns true on success
bool ExportWaveAsCode(Wave wave, const char *fileName);     // Export wave sample data to code (.h), returns true on success

// Wave/Sound management functions
void PlaySound(Sound sound);                                // Play a sound
void StopSound(Sound sound);                                // Stop playing a sound
void PauseSound(Sound sound);                               // Pause a sound
void ResumeSound(Sound sound);                              // Resume a paused sound
bool IsSoundPlaying(Sound sound);                           // Check if a sound is currently playing
void SetSoundVolume(Sound sound, float volume);             // Set volume for a sound (1.0 is max level)
void SetSoundPitch(Sound sound, float pitch);               // Set pitch for a sound (1.0 is base level)
void SetSoundPan(Sound sound, float pan);                   // Set pan for a sound (-1.0 left, 0.0 center, 1.0 right)
Wave WaveCopy(Wave wave);                                   // Copy a wave to a new wave
void WaveCrop(Wave *wave, int initFrame, int finalFrame);   // Crop a wave to defined frames range
void WaveFormat(Wave *wave, int sampleRate, int sampleSize, int channels); // Convert wave data to desired format
float *LoadWaveSamples(Wave wave);                          // Load samples data from wave as a 32bit float data array
void UnloadWaveSamples(float *samples);                     // Unload samples data loaded with LoadWaveSamples()

// Music management functions
Music LoadMusicStream(const char *fileName);                // Load music stream from file
Music LoadMusicStreamFromMemory(const char *fileType, const unsigned char *data, int dataSize); // Load music stream from data
bool IsMusicValid(Music music);                             // Checks if a music stream is valid (context and buffers initialized)
void UnloadMusicStream(Music music);                        // Unload music stream
void PlayMusicStream(Music music);                          // Start music playing
bool IsMusicStreamPlaying(Music music);                     // Check if music is playing
void UpdateMusicStream(Music music);                        // Updates buffers for music streaming
void StopMusicStream(Music music);                          // Stop music playing
void PauseMusicStream(Music music);                         // Pause music playing
void ResumeMusicStream(Music music);                        // Resume playing paused music
void SeekMusicStream(Music music, float position);          // Seek music to a position (in seconds)
void SetMusicVolume(Music music, float volume);             // Set volume for music (1.0 is max level)
void SetMusicPitch(Music music, float pitch);               // Set pitch for a music (1.0 is base level)
void SetMusicPan(Music music, float pan);                   // Set pan for a music (-1.0 left, 0.0 center, 1.0 right)
float GetMusicTimeLength(Music music);                      // Get music time length (in seconds)
float GetMusicTimePlayed(Music music);                      // Get current music time played (in seconds)

// AudioStream management functions
AudioStream LoadAudioStream(unsigned int sampleRate, unsigned int sampleSize, unsigned int channels); // Load audio stream (to stream raw audio pcm data)
bool IsAudioStreamValid(AudioStream stream);                // Checks if an audio stream is valid (buffers initialized)
void UnloadAudioStream(AudioStream stream);                 // Unload audio stream and free memory
void UpdateAudioStream(AudioStream stream, const void *data, int frameCount); // Update audio stream buffers with data
bool IsAudioStreamProcessed(AudioStream stream);            // Check if any audio stream buffers requires refill
void PlayAudioStream(AudioStream stream);                   // Play audio stream
void PauseAudioStream(AudioStream stream);                  // Pause audio stream
void ResumeAudioStream(AudioStream stream);                 // Resume audio stream
bool IsAudioStreamPlaying(AudioStream stream);              // Check if audio stream is playing
void StopAudioStream(AudioStream stream);                   // Stop audio stream
void SetAudioStreamVolume(AudioStream stream, float volume);    // Set volume for audio stream (1.0 is max level)
```

```
void SetAudioStreamPitch(AudioStream stream, float pitch);      // Set pitch for audio stream (1.0 is base level)
void SetAudioStreamPan(AudioStream stream, float pan);          // Set pan for audio stream (-1.0 to 1.0 range, 0.0 is centered)
void SetAudioStreamBufferSizeDefault(int size);                 // Default size for new audio streams
void SetAudioStreamCallback(AudioStream stream, AudioCallback callback); // Audio thread callback to request new data

void AttachAudioStreamProcessor(AudioStream stream, AudioCallback processor); // Attach audio stream processor to stream, receives frames x 2 samples as 'float' (stereo)
void DetachAudioStreamProcessor(AudioStream stream, AudioCallback processor); // Detach audio stream processor from stream

void AttachAudioMixedProcessor(AudioCallback processor); // Attach audio stream processor to the entire audio pipeline, receives frames x 2 samples as 'float' (stereo)
void DetachAudioMixedProcessor(AudioCallback processor); // Detach audio stream processor from the entire audio pipeline
```

structs

```
struct Vector2;                // Vector2, 2 components
struct Vector3;                // Vector3, 3 components
struct Vector4;                // Vector4, 4 components
struct Matrix;                 // Matrix, 4x4 components, column major, OpenGL style, right-handed
struct Color;                  // Color, 4 components, R8G8B8A8 (32bit)
struct Rectangle;              // Rectangle, 4 components

struct Image;                  // Image, pixel data stored in CPU memory (RAM)
struct Texture;                // Texture, tex data stored in GPU memory (VRAM)
struct RenderTexture;          // RenderTexture, fbo for texture rendering
struct NPatchInfo;             // NPatchInfo, n-patch layout info
struct GlyphInfo;              // GlyphInfo, font characters glyphs info
struct Font;                   // Font, font texture and GlyphInfo array data

struct Camera3D;               // Camera, defines position/orientation in 3d space

struct Camera2D;               // Camera2D, defines position/orientation in 2d space
struct Mesh;                   // Mesh, vertex data and vao/vbo
struct Shader;                 // Shader
struct MaterialMap;            // MaterialMap
struct Material;               // Material, includes shader and maps
struct Transform;              // Transform, vertex transformation data
struct BoneInfo;               // Bone, skeletal animation bone
struct ModelSkeleton;          // Skeleton, animation bones hierarchy
struct Model;                  // Model, meshes, materials and animation data
struct ModelAnimation;         // ModelAnimation, contains a full animation sequence
struct Ray;                    // Ray, ray for raycasting
struct RayCollision;           // RayCollision, ray hit information
struct BoundingBox;            // BoundingBox

struct Wave;                   // Wave, audio wave data
struct AudioStream;            // AudioStream, custom audio stream
struct Sound;                  // Sound
struct Music;                  // Music, audio stream, anything longer than ~10 seconds should be streamed

struct VrDeviceInfo;           // VrDeviceInfo, Head-Mounted-Display device parameters
struct VrStereoConfig;         // VrStereoConfig, VR stereo rendering configuration for simulator

struct FilePathList;           // File path list
struct AutomationEvent;        // Automation event
struct AutomationEventList;    // Automation event list
```

colors

```
// Custom raylib color palette for amazing visuals on WHITE background
#define LIGHTGRAY  (Color){ 200, 200, 200, 255 }   // Light Gray
#define GRAY       (Color){ 130, 130, 130, 255 }   // Gray
#define DARKGRAY   (Color){ 80, 80, 80, 255 }      // Dark Gray
#define YELLOW     (Color){ 253, 249, 0, 255 }     // Yellow
#define GOLD       (Color){ 255, 203, 0, 255 }     // Gold
#define ORANGE     (Color){ 255, 161, 0, 255 }     // Orange
#define PINK       (Color){ 255, 109, 194, 255 }   // Pink
#define RED        (Color){ 230, 41, 55, 255 }     // Red
#define MAROON     (Color){ 190, 33, 55, 255 }     // Maroon
#define GREEN      (Color){ 0, 228, 48, 255 }      // Green
#define LIME       (Color){ 0, 158, 47, 255 }      // Lime
#define DARKGREEN  (Color){ 0, 117, 44, 255 }      // Dark Green
#define SKYBLUE    (Color){ 102, 191, 255, 255 }   // Sky Blue
#define BLUE       (Color){ 0, 121, 241, 255 }     // Blue
#define DARKBLUE   (Color){ 0, 82, 172, 255 }      // Dark Blue
#define PURPLE     (Color){ 200, 122, 255, 255 }   // Purple
#define VIOLET     (Color){ 135, 60, 190, 255 }    // Violet
#define DARKPURPLE (Color){ 112, 31, 126, 255 }    // Dark Purple
#define BEIGE      (Color){ 211, 176, 131, 255 }   // Beige
#define BROWN      (Color){ 127, 106, 79, 255 }    // Brown
#define DARKBROWN  (Color){ 76, 63, 47, 255 }      // Dark Brown

#define WHITE      (Color){ 255, 255, 255, 255 }   // White
#define BLACK      (Color){ 0, 0, 0, 255 }         // Black
#define BLANK      (Color){ 0, 0, 0, 0 }           // Blank (Transparent)
#define MAGENTA    (Color){ 255, 0, 255, 255 }     // Magenta
#define RAYWHITE   (Color){ 245, 245, 245, 255 }   // My own White (raylib logo)
```

Other cheatsheets

- raymath cheatsheet

raylib quick reference card - Copyright (c) 2013-2026 Ramon Santamaria (@raysan5)